Jira Enhancer: Supplementary Text
Appendices C and D

Anonymous authors

C Scenario-Level Transaction Outcomes

Table C.8 expands the aggregate transaction results in Table 5b of the main manuscript. Each scenario–arrangement cell contains 2,000 observations. With random seed 20260726, the same shuffled schedule of 18,000 transactions is evaluated under each arrangement, giving 72,000 method-level observations. The valid-control and two retry scenarios provide 6,000 eligible-write observations per arrangement. The other six scenarios provide 12,000 conflicting-state observations per arrangement.

Table C.8: Complete scenario-level transaction matrix. Each cell reports (m, s, d): mean modeled mutations per transaction, stale-mutation flag, and duplicate-mutation flag. The latter two entries are 0 or 1 because their rates are 0% or 100% in every cell.

Scenario	A0	A1	A2	A3
Valid control	$(1, 0, 0)$	$(1, 0, 0)$	$(1, 0, 0)$	$(1, 0, 0)$
Source edit	$(1, 1, 0)$	$(0, 0, 0)$	$(0, 0, 0)$	$(0, 0, 0)$
Source ABA	$(1, 1, 0)$	$(1, 1, 0)$	$(1, 1, 0)$	$(0, 0, 0)$
Draft drift	$(1, 1, 0)$	$(1, 1, 0)$	$(1, 1, 0)$	$(0, 0, 0)$
Policy drift	$(1, 1, 0)$	$(1, 1, 0)$	$(1, 1, 0)$	$(0, 0, 0)$
Target drift	$(1, 1, 0)$	$(1, 1, 0)$	$(1, 1, 0)$	$(0, 0, 0)$
Response-loss retry	$(2, 0, 1)$	$(2, 0, 1)$	$(1, 0, 0)$	$(1, 0, 0)$
Concurrent retry (modeled)	$(2, 0, 1)$	$(2, 0, 1)$	$(1, 0, 0)$	$(1, 0, 0)$
Combined source/draft drift	$(2, 1, 1)$	$(0, 0, 0)$	$(0, 0, 0)$	$(0, 0, 0)$

A0 uses ordinary approval and logging; A1 adds a source-hash check; A2 adds atomic-idempotency behavior; A3 adds the source-bound transaction envelope. The executable harness calls the implementation's A3 envelope builder and validator. It models the resulting mutation count as zero when blocked, one for an allowed A2/A3 transaction, and the attempt count for an allowed A0/A1 transaction. Thus, the matrix measures validator behavior coupled to declared mutation semantics, rather than observed remote Jira writes. The concurrent-retry scenario is represented by two attempts; the harness does not execute concurrent requests. Seven separate workflow tests exercise the mock connector, including completed-write retries and recovery after a lost commit response. These tests complement the scenario matrix without establishing production fault frequencies or concurrent-load performance.

D Downstream Rebalancing Implementation Case

D.1 Case provenance and staged binding distribution

The author identifies one Jira Enhancer-derived enhancement spanning three implementation stories: the subscriber service, Java SDK, and Python SDK. The execution evidence below covers the subscriber service and Python SDK. The companion story-case manifest maps the three components to the corresponding execution evidence.

The recovered execution archives contain pooling-enabled and pooling-disabled runs from 15 October 2026 in the development cloud cell. An independent artifact check recomputed all 19 declared file hashes in each archive and inspected the raw observations, stage snapshots, and handoff sequences.

Each run used five persistent Python clients with 100, 200, 500, 1,000, and 1,500 subscriptions, respectively: 3,300 client–subscription bindings across 1,500 distinct logical subscription identifiers. Pods were manually scaled from one to two, three, and four. Every observed scale-out stage met the 420-second minimum hold. Table D.9 reports final ready-binding snapshots. Pod aliases denote creation order separately within each run.

Table D.9: Ready client–subscription bindings and minimum worker hold after each manual scale-out stage. Each row totals 3,300 bindings; a dash indicates that the pod had not yet been created. The required minimum hold was 420 seconds.

Pooling	Pods	P1	P2	P3	P4	Hold (s)
Enabled	2	1,860	1,440	–	–	456.9
Enabled	3	1,240	1,150	910	–	444.8
Enabled	4	930	840	810	720	442.3
Disabled	2	1,700	1,600	–	–	443.2
Disabled	3	800	1,500	1,000	–	433.3
Disabled	4	500	1,500	1,000	300	439.3

D.2 Handoff records, telemetry, and interpretation

Table D.10 summarizes the execution records. Completed handoffs recorded these phases: `handshake`, `promote`, `ready`, `commit`, `quiesce`, `drain`, `retire_authorized`, and `retire`. The checks correlate committed target ownership with same-session subscription receipts and source `DRAINED` and `RETIRED` wire responses. The enabled archive contains 14 complete sequences (112 phase records), and the disabled archive contains six (48 records), with no additional incomplete sequence in the inspected handoff observations. Five disabled-run handoffs targeted newly created pods; these account for 2,900 of its 3,000 cumulative binding moves. Movement counts sum bindings transferred across handoffs.

Table D.10: Descriptive execution and non-probe MainStats telemetry from the retained runs. Cadence is measured between consecutive frames within the same client, physical connection, and session.

Recorded quantity	Enabled	Disabled
Complete handoffs / phase records	14 / 112	6 / 48
Cumulative binding moves	3,070	3,000
Non-probe MainStats frames	2,476	933
Connection–session groups	27	11
Within-group arrival intervals	2,449	922
Median interval (seconds)	10.000391	10.000380
Maximum interval (seconds)	10.205	10.208
Intervals above 11 seconds	0	0
Business events / ACKs / NACKs	0 / 0 / 0	0 / 0 / 0

All five Python workers exited with status zero in each run. The records verify subscription placement and receipt-gated handoffs. MainStats intervals measure telemetry cadence.

The archived stage snapshots, completed handoff sequences, and successful worker exits demonstrate downstream execution feasibility for this rebalancing enhancement.